

Assignment No. 2

Sanskruti Dahiphale – 123B1B007

1. Problem Statement

Design a greedy algorithm to maximize profit in a warehouse by selecting and transporting items to a single destination within a given weight limit. Each item has a specified weight and profit. Implement the algorithm and verify that it maximizes profit while staying within the weight constraint.

2. Prerequisite

1. To study the problem of profit maximization underweight constraints.
2. To understand and implement the Greedy Algorithm strategy.
3. To improve efficiency in solving optimization problems.
4. To verify that the solution satisfies the weight constraint while maximizing profit.

Course Outcome Mapping:

CO2: Analyze the efficiency of algorithm using either Divide and Conquer or Greedy strategy.

Theory:

In warehouse management systems, transportation vehicles have limited capacity. Each item stored in the warehouse has a specific weight and profit value. When transporting goods, selecting items randomly may not give maximum profit. Therefore, an optimized method is required.

The Greedy Algorithm is an approach that makes the best possible choice at each step. It selects the option that provides maximum immediate benefit. For this problem, the Fractional Knapsack Algorithm is used.

In the Fractional Knapsack problem, items can be taken fully or partially (fractionally). The key idea is to select items based on their profit-to-weight ratio.

Items with higher ratios are selected *Profit* first. This *Ratio* ensures = $\frac{\text{Weight} \text{ Profit}}{\text{Weight}}$ maximum profit per unit weight.

Greedy Algorithm (Fractional Knapsack):

Steps:

1. Input number of objects and capacity of sack.
2. For each object, input weight and profit.
3. Calculate profit-to-weight ratio for each object.
4. Sort all objects in descending order of ratio.
5. Select objects one by one:
 - o If the object completely fits, take it fully.
 - o If not, take the fractional part that fits.
6. Continue until the capacity becomes zero.

Example:

Number of objects = 7

Capacity of sack = 27

Object	Weight	Profit	Profit/Weight
1	3	25	8.33
2	11	55	5.00
3	5	40	8.00
4	2	15	7.50
5	8	90	11.25
6	3	35	11.67
7	6	65	10.83

Step 1: Sort by Profit/Weight Ratio (Descending)

Order becomes:

Object 6

Object 5

Object 7
Object 1
Object 3
Object 4
Object 2

Step 2: Select Items

Capacity = 27

Take Object 6 (weight 3, profit 35) Remaining capacity = 24

Take Object 5 (weight 8, profit 90) Remaining capacity = 16

Take Object 7 (weight 6, profit 65) Remaining capacity = 10

Take Object 1 (weight 3, profit 25) Remaining capacity = 7

Take Object 3 (weight 5, profit 40) Remaining capacity = 2

Take Object 4 (weight 2, profit 15)
Remaining capacity = 0

Total Profit = $35 + 90 + 65 + 25 + 40 + 15$
 $= 270$

Time Complexity:

- Calculating ratios $\rightarrow O(n)$
- Sorting $\rightarrow O(n \log n)$
- Selection process $\rightarrow O(n)$

Overall Time Complexity:

The sorting step dominates $O(n \log n)$ complexity.

Code:

Fractional Greedy Knapsack # Take

inputs n = int(input("Enter number of

```
objects: ")) m = float(input("Enter capacity  
of sack: "))
```

```
weights = [] profits  
= [] for i in  
range(n):
```

```
    w = float(input(f"Enter weight of object {i+1}: "))  
    p = float(input(f"Enter profit of object {i+1}: "))  
    weights.append(w)    profits.append(p)
```

```
# Compute profit/weight ratio
```

```
items = [] for i in range(n):  
    ratio = profits[i] / weights[i]  
    items.append((ratio, weights[i], profits[i]))
```

```
# Sort in descending order of ratio items.sort(reverse=True)
```

```
# Apply Greedy Selection
```

```
total_profit = 0 remaining_capacity  
= m
```

```
for ratio, weight, profit in items:
```

```
    if remaining_capacity == 0:
```

```
        break
```

```
        if weight <= remaining_capacity:
```

```
# Take full item
```

```
remaining_capacity -= weight
```

```
total_profit += profit    else:
```

```

        # Take fractional part      fraction =
remaining_capacity / weight
total_profit += profit * fraction
remaining_capacity = 0

# Output print("\nMaximum Profit =",
total_profit)

```

Output:

```

Enter number of items available in warehouse: 3
Enter maximum weight capacity of transport vehicle: 50

Enter weight of item 1: 10
Enter profit of item 1: 60

Enter weight of item 2: 20
Enter profit of item 2: 100

Enter weight of item 3: 30
Enter profit of item 3: 120

```

Output

```

Maximum Profit from transported items = 240

```

Conclusion:

The Greedy Algorithm using the Fractional Knapsack method successfully maximizes profit within the given weight constraint. By selecting items based on the highest profit-to-weight ratio, the algorithm ensures optimal utilization of capacity.

The time complexity is $O(n \log n)$, making it efficient for large inputs. The Greedy approach works effectively in this problem because fractional selection is allowed. Therefore, it provides an optimal and efficient solution for profit maximization in warehouse transportation systems.